



Semana 9 – Laboratorio: Optimización de Recursos mediante Programación Dinámica

ICC708-1: PROGRAMACION AVANZADA

Nombre: Bárbara Constanza Sanhueza Bombín.

Carrera: Ingeniería informática.

Fecha: 10-07-2026.

Docente: Profesor Ricardo Gacitúa B.

Introducción

En este laboratorio se aborda el problema de la Mochila 0/1 mediante programación dinámica. El objetivo es seleccionar un conjunto de objetos que permita obtener el mayor valor posible sin superar una capacidad máxima de peso.

Cada objeto posee un nombre, un peso y un valor asociado. La condición 0/1 indica que cada elemento puede seleccionarse una sola vez o descartarse. La solución implementada construye una tabla dinámica, obtiene el valor máximo, reconstruye los objetos seleccionados y muestra el peso y valor total de la solución. Además, el programa realiza experimentos con distintas cantidades de objetos generados aleatoriamente, registrando datos como el tiempo de ejecución, la capacidad utilizada, el tamaño de la tabla y el valor óptimo encontrado.

Descripción del problema

El problema representa a una organización de ayuda humanitaria que debe transportar suministros en un vehículo con capacidad limitada. Entre los recursos disponibles se encuentran agua, medicamentos, alimentos, radios, generadores y herramientas.

Cada recurso posee un peso y un valor de prioridad. Debido a que no todos los objetos pueden transportarse, se debe encontrar la combinación que maximice el valor total sin superar la capacidad del vehículo.

Para la ejecución inicial se utiliza una capacidad máxima de 20 unidades y un conjunto de seis objetos. Cada objeto puede ser seleccionado como máximo una vez.

Diseño de la solución

La solución fue desarrollada con las siguientes clases; la clase `Objeto` representa cada recurso y almacena su nombre, peso y valor. La clase `Mochila01` contiene el algoritmo principal, la reconstrucción de la solución, la impresión de la tabla dinámica y la ejecución de los experimentos.

El método `resolverMochila` crea una tabla bidimensional y evalúa, para cada objeto y capacidad disponible, si conviene incluir el objeto actual o excluirlo. En cada posición se almacena la opción que entregue el mayor valor.

Una vez completada la tabla, el valor ubicado en la última celda representa el valor máximo alcanzable. Luego, el programa recorre la tabla en sentido inverso para identificar los objetos seleccionados y calcular su peso y valor total.

El método `calcularMochila` utiliza la misma lógica, pero retorna solamente el valor óptimo. Este método se emplea en los experimentos para evitar imprimir y reconstruir la solución en cada ejecución.

Descripción de estados y recurrencia utilizada

El estado de la solución se representa mediante $dp[i][w]$, donde:

- i indica la cantidad de objetos considerados.
- w representa la capacidad disponible.
- $dp[i][w]$ almacena el valor máximo alcanzable con los primeros i objetos y una capacidad w .

Los casos base se encuentran en la fila cero y en la columna cero. Si no hay objetos disponibles o la capacidad es cero, el valor máximo es cero.

Si el peso del objeto actual supera la capacidad disponible, el objeto no puede incluirse:

$$dp[i][w] = dp[i - 1][w]$$

Si el objeto cabe en la mochila, se comparan dos alternativas: excluirlo o incluirlo.

$$dp[i][w] = \text{máximo}(dp[i - 1][w], \text{valor}[i] + dp[i - 1][w - \text{peso}[i]])$$

La primera opción mantiene el valor anterior. La segunda suma el valor del objeto actual al mejor resultado disponible para la capacidad restante. El algoritmo conserva la alternativa con mayor valor.

Resultados

Tabla de programación dinámica

La tabla DP generada contiene una fila por cada objeto considerado y una columna por cada capacidad posible, desde 0 hasta 20. Cada celda representa el valor máximo que puede alcanzarse utilizando los objetos disponibles hasta esa fila.

En la fila inicial, denominada Ninguno, todos los valores son cero, debido a que no se ha considerado ningún objeto. A medida que se incorporan los recursos, los valores aumentan cuando una nueva combinación mejora la solución anterior. La última celda de la tabla, correspondiente a todos los objetos y una capacidad de 20, contiene el valor **39**, confirmando el resultado óptimo obtenido por el algoritmo.

Objeto / Capacidad	0	2	5	10	15	20
Ninguno	0	0	0	0	0	0
Agua	0	6	6	6	6	6
Medicamentos	0	6	14	14	14	14

Alimentos	0	6	14	21	21	21
Radio	0	6	14	24	31	31
Generador	0	6	14	24	31	39
Herramientas	0	6	14	24	31	39

La tabla anterior corresponde a una versión resumida de la tabla completa, mostrando algunas capacidades representativas para facilitar su visualización.

Resultados experimentales

Para evaluar el comportamiento del algoritmo se generaron conjuntos aleatorios de 10, 20, 50, 100 y 200 objetos. En cada caso se registró la capacidad de la mochila, el tiempo de ejecución, el tamaño de la tabla DP y el valor óptimo obtenido.

Objetos	Capacidad	Tiempo de ejecución (ns)	Tamaño de la tabla DP	Valor óptimo
10	50	69.900	561	497
20	100	135.600	2.121	784
50	250	727.000	12.801	2.105
100	500	2.910.800	50.601	3.492
200	1.000	1.537.100	201.201	8.544

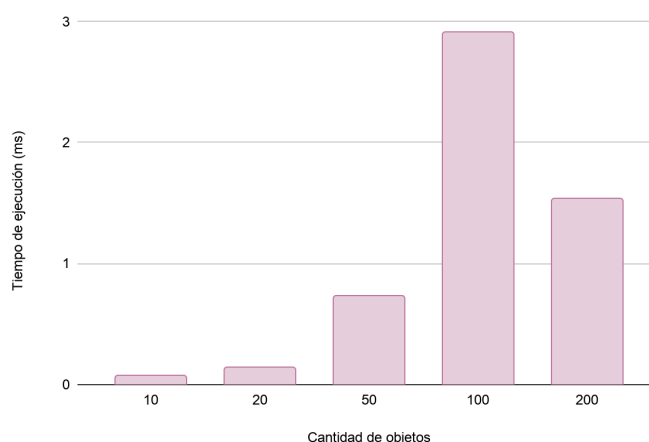
Los resultados muestran que el tamaño de la tabla aumenta considerablemente al incrementar la cantidad de objetos y la capacidad de la mochila. El valor óptimo también tiende a aumentar, debido a que se dispone de una mayor cantidad de objetos y de una capacidad superior.

El tiempo de ejecución presentó un crecimiento general hasta el experimento de 100 objetos. Sin embargo, la ejecución con 200 objetos registró un tiempo menor. Esta variación puede producirse por factores propios de una medición única, como la optimización de la máquina virtual de Java, la carga del sistema o la precisión de `System.nanoTime()`.

Gráficos obtenidos

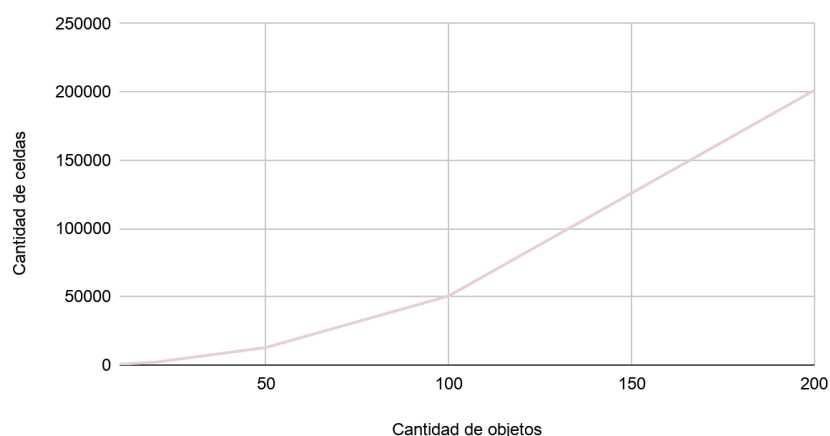
A partir de los resultados experimentales se pueden construir los siguientes gráficos:

Tiempo de ejecución según la cantidad de objetos



El tiempo de ejecución aumentó desde 0,0699 ms para 10 objetos hasta 2,9108 ms para 100 objetos. Sin embargo, la ejecución con 200 objetos registró 1,5371 ms, un valor menor al caso anterior. Esta variación puede deberse a que se realizó una sola medición y a factores como la optimización de la máquina virtual de Java o la carga del sistema durante la ejecución.

Tamaño de la tabla DP según la cantidad de objetos



El tamaño de la tabla dinámica aumentó de 561 celdas para 10 objetos a 201.201 celdas para 200 objetos. Este crecimiento se debe a que la tabla necesita una fila por cada objeto considerado y una columna por cada capacidad posible de la mochila.

Análisis

Discusión de complejidad observada

El algoritmo utiliza una tabla de tamaño aproximado $(n + 1) \times (W + 1)$, donde n es la cantidad de objetos y W la capacidad de la mochila. Por ello, su complejidad temporal es **$O(nW)$** .

Los experimentos muestran un aumento general del tiempo y del tamaño de la tabla al crecer la entrada. La única variación irregular aparece con 200 objetos, cuyo tiempo fue menor que el caso de 100 objetos. Esto puede deberse a que se realizó una sola medición y a factores como la optimización de Java o la carga del sistema.

Preguntas y respuestas

¿Cómo crece el tiempo de ejecución al aumentar el número de objetos?

El tiempo tiende a aumentar, porque el algoritmo debe procesar más filas de la tabla DP. En los resultados, el crecimiento fue visible hasta 100 objetos, aunque el caso de 200 presentó una medición menor por posibles variaciones del entorno.

¿Cómo afecta la capacidad máxima de la mochila al tiempo de ejecución?

Una capacidad mayor genera más columnas en la tabla, por lo que se deben calcular más posiciones para cada objeto. Por esta razón, el tiempo de ejecución también aumenta.

¿Coinciden los resultados observados con la complejidad teórica $O(nW)$?

Sí, de forma general. El tamaño de la tabla pasó de 561 a 201.201 celdas al aumentar los objetos y la capacidad. La irregularidad en los tiempos no contradice la complejidad teórica, ya que corresponde a mediciones puntuales.

¿Qué ventajas presenta la Programación Dinámica respecto de una búsqueda exhaustiva?

La programación dinámica almacena y reutiliza resultados parciales, evitando repetir cálculos. Esto permite resolver el problema de forma más eficiente que una búsqueda exhaustiva, la cual tendría que evaluar todas las combinaciones posibles de objetos.

Conclusiones.

La implementación permitió resolver correctamente el problema de la Mochila 0/1 mediante programación dinámica, obteniendo la combinación de objetos con mayor valor sin superar la capacidad máxima. A partir de este laboratorio se comprendió cómo definir estados, aplicar una recurrencia y reconstruir una solución óptima utilizando una tabla DP.

Los experimentos mostraron que el tamaño de la tabla aumenta al crecer la cantidad de objetos y la capacidad de la mochila, lo que coincide de forma general con la complejidad teórica $O(nW)$. También se observó que los tiempos de ejecución pueden variar entre pruebas, por lo que resulta recomendable realizar varias mediciones y utilizar un promedio.

En comparación con una búsqueda exhaustiva, la programación dinámica permite reutilizar resultados parciales y evitar cálculos repetidos, ofreciendo una solución más eficiente para este tipo de problemas de optimización.